Week 8: Asynchronous HTTP and Database Connections in Flutter

Futures, REST APIs, SQLite, repositories and offline-first apps

Dr Mohamad AOUDE

Introduction to Multiplatform – Semester VIII
Lebanese University – Faculty of Engineering, Branch II

The challenge: waiting without freezing

- A student taps extbfLoad Tasks; the server responds three seconds later.
- The user should still be able to scroll, tap and see an animated loading indicator.
- We will build a extbfStudent Task Manager using remote HTTP and local SQLite.

Big idea

HTTP moves data between systems; a database persists data. Asynchronous Dart coordinates both without blocking the UI.

Discuss. What should happen during those three seconds?

What you will build

- Retrieve tasks using GET and decode JSON.
- Display loading, success, empty and error states.
- Submit a POST request to a safe test API.
- Persist tasks in a local SQLite database.
- Show cached tasks when the network is unavailable.
- Test failures, timeouts and repository behavior.

Try it now. Before coding, predict which operations must finish before the next can start.

Roadmap and pacing

Module	Focus	Minutes
A	Async mental model and Dart	25
B	HTTP, JSON and UI	30
C	SQLite and local persistence	25
D	Repository and offline-first sync	25
E	Robustness, tests and capstone	25

Suggested format: two 60–70 minute sessions or a longer workshop.

A. Async foundations

The language shared by HTTP and databases

Synchronous vs asynchronous work

Blocking work

Task A $\rightarrow$ wait $\rightarrow$ Task B
UI cannot process other events during a long synchronous operation.

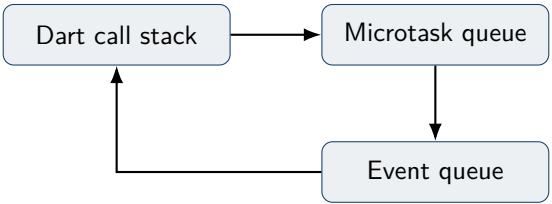
Nonblocking I/O

Start A $\rightarrow$ process other events $\rightarrow$ resume A
The pending operation completes later.

Key distinction: `await` suspends an async function, not the entire UI isolate.

Discuss. Does marking a heavy computation `async` make it nonblocking? Why not?

The event loop and the UI isolate



Each isolate has its own event loop. Microtasks run before the next event; excessive microtasks can starve events. OS/network/database mechanisms complete I/O outside ordinary synchronous Dart execution.

Discuss. Why does a five-second HTTP request not necessarily freeze the screen?

A Future is a result that arrives later

- `Future<T>` eventually completes with a value of type `T` or an error.
- `Future<void>` reports completion without a returned value.
- `async` makes the function return a Future; `await` obtains its eventual result.
- A Future has one completion; a Stream can emit multiple values.

```dart
Future<String> fetchName() async {
  await Future.delayed(const Duration(seconds: 2));
  return "Amina";
}
```

Discuss. Is the function returning a String immediately?

Predict the output

```dart
Future<void> load() async {
  print("1. Start");
  await Future.delayed(const Duration(seconds: 2));
  print("2. Completed");
}

void main() {
  load();
  print("3. Still running");
}
```

Prediction

Write the output order before running the program. Then explain why the result is 1, 3, 2.

Callback style and async/await

```dart
Future<String> getTitle() => Future.value("Hello");

void callbacks() {
  getTitle().then((title) => print(title));
}

Future<void> withAwait() async {
  final title = await getTitle();
  print(title);
}
```

Teaching rule

Prefer async/await for readable sequential control flow; understand then/catchError when reading APIs.

Sequential vs concurrent

```dart
Future<String> task(String label) async {
  await Future.delayed(const Duration(seconds: 2));
  return label;
}

// Sequential: about four seconds
final first = await task("A");
final second = await task("B");

// Concurrent: about two seconds
final results = await Future.wait([task("A"), task("B")]);
```

Discuss. Which version is valid if task B needs the result of task A?

Errors and finally

```
Future<void> load() async {
  try {
    final data = await fetchRemoteData();
    print(data);
  } on TimeoutException {
    print("Request timed out");
  } on FormatException {
    print("Invalid response format");
  } catch (error) {
    print("Unexpected: $error");
  } finally {
    print("Finished attempting load");
  }
}
```

Imports and `fetchRemoteData()` are supplied by the surrounding application.

Discuss. Does finally mean the request succeeded?

Future vs Stream

	Future	Stream
Values	One completion	Zero or more events
Typical use	HTTP GET, SQLite insert	WebSocket, reactive DB watch
Flutter widget	FutureBuilder	StreamBuilder
Error handling	try/catch with await	onError / async* / listen

Important

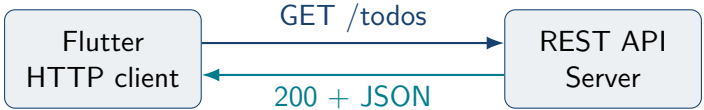
SQLite via `sqflite` returns Futures; it does not automatically provide reactive watch streams. Drift supports watched queries.

Discuss. Which would you use for a live chat feed?

B. HTTP connections

From a URL to a Flutter widget

HTTP request/response architecture



Request: method, URL, headers, optional body. Response: status code, headers, optional body.

Discuss. Which side chooses the HTTP status code?

HTTP methods and status codes

Method	Meaning	Status	Typical meaning
GET	Read a resource	200	OK
POST	Create or submit	201	Created
PUT	Replace resource	400	Bad request
PATCH	Partially modify	401/403	Auth / forbidden
DELETE	Delete resource	404	Not found
		429	Rate limited
		500/503	Server issue

Reminder

A 404 is an HTTP response. A DNS failure or disconnected network may produce no HTTP response at all.

Safe public HTTP playgrounds

Service	Example endpoint	Best experiment
JSONPlaceholder	`jsonplaceholder.typicode.com/todos`	GET and simulated POST/PUT/DELETE
httpbin	`httpbin.org/status/404`	Status, headers and delays
DummyJSON	`dummyjson.com/products`	Product catalog and search

Classroom caution

These services are public test tools. Do not send credentials or personal data. JSONPlaceholder writes are not persistent.

Try it now. Open `https://jsonplaceholder.typicode.com/todos/1` and identify its JSON fields.

Project setup

```
flutter create student_task_manager
cd student_task_manager
flutter pub add http sqflite path
```

Put `<uses-permission android:name="android.permission.INTERNET"/>` inside the Android manifest element (for the application build). Use HTTPS endpoints.

Suggested files: `models/task.dart`, `data/task_api.dart`, `data/task_dao.dart`, `data/task_repository.dart`, `main.dart`.

Try it now. Create the project and verify that an HTTPS endpoint opens in the browser.

GET: request and decode

```dart
import 'dart:convert';
import 'package:http/http.dart' as http;

Future<List<dynamic>> fetchTodos() async {
  final uri = Uri.https(
    'jsonplaceholder.typicode.com', '/todos');
  final res = await http.get(uri)
      .timeout(const Duration(seconds: 8));
  if (res.statusCode != 200) {
    throw Exception('HTTP ${res.statusCode}');
  }
  return jsonDecode(res.body) as List<dynamic>;
}
```

Discuss. Why check the status code before decoding the expected JSON?

Typed Task model

```dart
class Task {
  const Task({required this.id, required this.title,
              required this.completed});
  final int id;
  final String title;
  final bool completed;

  factory Task.fromJson(Map<String, dynamic> json) {
    return Task(
      id: json['id'] as int,
      title: json['title'] as String,
      completed: json['completed'] as bool,
    );
  }
}
```

Discuss. What exception might occur if the server changes a field type?

HTTP service with an injectable client

```dart
class TaskApi {
  TaskApi(this.client);
  final http.Client client;

  Future<List<Task>> fetchTasks() async {
    final res = await client.get(Uri.https(
      'jsonplaceholder.typicode.com', '/todos'))
      .timeout(const Duration(seconds: 8));
    if (res.statusCode != 200) {
      throw Exception('HTTP ${res.statusCode}');
    }
    final rows = jsonDecode(res.body) as List<dynamic>;
    return rows.map((e) => Task.fromJson(
      e as Map<String, dynamic>)).toList();
  }
}
```

Imports and the Task model are as defined earlier. Client injection enables deterministic HTTP tests.

FutureBuilder and its lifecycle

```
late Future<List<Task>> futureTasks;

@override
void initState() {
  super.initState();
  futureTasks = api.fetchTasks();
}

// In build():
FutureBuilder<List<Task>>(
  future: futureTasks,
  builder: (context, snapshot) {
    if (!snapshot.hasData) {
      if (snapshot.hasError) return const Text('Load failed');
      return const CircularProgressIndicator();
    }
    return Text('${snapshot.data!.length} tasks');
  },
)
```

Discuss. Why not call api.fetchTasks() directly inside build()?

POST with JSON body

```dart
Future<int> createPost(http.Client client, String title) async {
  final res = await client.post(
    Uri.https('jsonplaceholder.typicode.com', '/posts'),
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode({
      'title': title, 'body': 'Created in Flutter',
      'userId': 1,
    }),
  );
  if (res.statusCode != 201) {
    throw Exception('HTTP ${res.statusCode}');
  }
  return (jsonDecode(res.body) as Map<String, dynamic>)
      ['id'] as int;
}
```

Crucial distinction

JSONPlaceholder simulates successful creation; retrieving the new ID later will not prove persistence.

HTTP failure laboratory

Endpoint	Student observation
`httpbin.org/status/200`	Success response
`httpbin.org/status/404`	Missing resource
`httpbin.org/status/500`	Server-side error
`httpbin.org/delay/3`	Artificial delay; compare with timeout
Invalid hostname	Transport exception; no HTTP status

Try it now. Predict the outcome, run each endpoint, then classify the failure as HTTP, transport, timeout or parsing.

Lab 1: HTTP task viewer

- Retrieve `/todos/1`, then `/todos/5`.
- Render task title and completion status.
- Show a progress indicator during the request.
- Handle missing resources without crashing.
- Provide a Refresh button that creates a new Future on demand.

Deliverable

A screenshot of the loaded task and a screenshot of a handled failure.

C. Database connections

Persistent local storage with SQLite

HTTP is not a database

	Remote API	Local SQLite
Primary job	Communicate with service	Persist structured data
Data location	Remote server	Device
Typical API	`http.get()`	`db.query()`
Result	Future of HTTP response	Future of database rows
Offline	May be unavailable	Normally accessible locally

Security rule

Do not embed production MySQL/PostgreSQL server credentials in a Flutter client. Use a protected backend API or a properly secured client SDK.

Open SQLite asynchronously

```dart
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';

Future<Database> openTaskDb() async {
  final path = join(await getDatabasesPath(), 'tasks.db');
  return openDatabase(path, version: 1,
    onCreate: (db, version) async {
      await db.execute('''
        CREATE TABLE tasks (
          id INTEGER PRIMARY KEY,
          title TEXT NOT NULL,
          completed INTEGER NOT NULL
        )
      ''');
    },
  );
}
```

Discuss. When is onCreate called? What is needed for schema changes?

Save a Task using a DAO

```
Future<void> saveTask(Database db, Task task) async {
  await db.insert(
    'tasks',
    {
      'id': task.id,
      'title': task.title,
      'completed': task.completed ? 1 : 0,
    },
    conflictAlgorithm: ConflictAlgorithm.replace,
  );
}
```

Database semantics

The returned Future completes after the insert operation. Conflict replacement is a deliberate upsert policy, not automatic synchronization.

Read and reconstruct Tasks

```dart
Future<List<Task>> readTasks(Database db) async {
  final rows = await db.query('tasks', orderBy: 'id');
  return rows.map((row) => Task(
    id: row['id'] as int,
    title: row['title'] as String,
    completed: (row['completed'] as int) == 1,
  )).toList();
}
```

Try it now. Insert three tasks, restart the application, and retrieve them from SQLite without an HTTP request.

Parameterized SQL and safe queries

```dart
final rows = await db.query(
  'tasks',
  where: 'id = ?',
  whereArgs: [taskId],
);

// Avoid building SQL from untrusted strings:
// db.rawQuery("SELECT * FROM tasks WHERE title = '$input'");
```

Security habit

Use placeholders and arguments for user-supplied values. Do not interpolate input into SQL statements.

CRUD: update, delete and transactions

```dart
await db.update('tasks', {'completed': 1},
  where: 'id = ?', whereArgs: [id]);

await db.delete('tasks',
  where: 'id = ?', whereArgs: [id]);

await db.transaction((txn) async {
  for (final task in tasks) {
    await txn.insert('tasks', task.toMap(),
      conflictAlgorithm: ConflictAlgorithm.replace);
  }
});
```

Here `task.toMap()` is a model helper students implement. Use `txn` rather than the outer database handle inside the transaction.

Discuss. Why is a transaction preferable for a batch of related writes?

Database alternatives and reactive data

Library	Programming model	Classroom use
sqflite	Future-based SQLite operations	Core local database lesson
drift	Typed SQL and watched query Streams	Reactive database extension
sembast	Async document/key-value store	Alternative storage model
Isar / Hive	Package/version-specific APIs	Optional ecosystem comparison
Supabase	Hosted Postgres via client API	Remote persistence extension

Do not confuse

A `Future<List<Task>>` is a snapshot; it does not automatically update the UI when the table changes.

Lab 2: real persistence

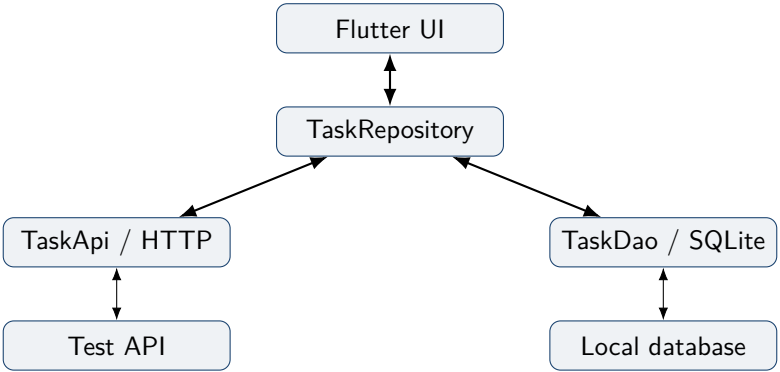
- Create a local SQLite database.
- Save tasks from the HTTP API to the local table.
- Read tasks using `db.query()`.
- Toggle a task completion flag and save it.
- Restart the app and verify that data remains.

Try it now. Turn off the internet before reopening the task list. Which layer still works?

D. HTTP + database integration

One repository, two asynchronous boundaries

Repository architecture



UI depends on repository methods, not on HTTP and SQL details.

Discuss. Where should the offline fallback policy live?

Repository: network-first, cache fallback

```
class TaskRepository {
  TaskRepository(this.api, this.db);
  final TaskApi api;
  final Database db;

  Future<List<Task>> load() async {
    try {
      final fresh = await api.fetchTasks();
      await db.transaction((txn) async {
        for (final task in fresh) {
          await txn.insert('tasks', task.toMap(),
            conflictAlgorithm: ConflictAlgorithm.replace);
        }
      });
      return fresh;
    } catch (_) {
      return readTasks(db);
    }
  }
}
```

Simplified example: distinguish network and database errors in production; an empty cache should not be misreported as a successful refresh.

Cache-first vs network-first

Strategy	Advantage	Trade-off
Network-first, fallback	Fresh data preferred	Slow initial screen online
Cache-first, refresh	Fast initial display	Cached data may be stale
Write-through/outbox	Can support offline edits	Needs conflict resolution

Teaching recommendation

Start with network-first to understand error fallback; then introduce cache-first as an advanced design.

Discuss. Which strategy is preferable for a weather forecast? For saved notes?

Cache-first flow

1. Open SQLite and display cached tasks.
2. Start an asynchronous HTTP refresh.
3. Validate response and map JSON to Task objects.
4. Persist the new records in a transaction.
5. Reload the UI or notify listeners.
6. On network failure, retain cached data and show an offline indicator.

Engineering caveat

A `sqflite` query does not automatically emit a Stream. Use explicit state notification or a reactive database library.

Race conditions and lifecycle

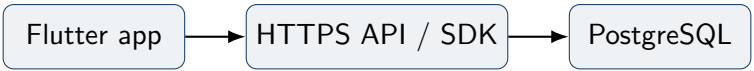
```
int requestVersion = 0;

Future<void> refresh() async {
  final version = ++requestVersion;
  final tasks = await repository.load();
  if (!mounted || version != requestVersion) return;
  setState(() => visibleTasks = tasks);
}
```

This is a widget-state example. A version guard prevents an older request from overwriting a newer UI result; it does not cancel the network operation or prevent repository-level stale writes.

Discuss. What happens if the user navigates away during await?

Remote database: where HTTP meets SQL



Supabase is one optional hosted Postgres example. A client SDK can communicate with a managed API; do not expose database administrator credentials in the app.

Security requirements

Use Row Level Security and appropriate user authentication. A public client key is not a substitute for authorization. Never bundle a service-role key.

Optional Supabase demonstration

```dart
// Illustrative code after configuring supabase_flutter.
final rows = await Supabase.instance.client
    .from('todos')
    .select()
    .order('id');

await Supabase.instance.client
    .from('todos')
    .insert({'title': 'Study async', 'done': false});
```

Requires a configured project, table, permissions and RLS policies. Do not use a production database in class.

Discuss. Which credentials are safe to expose to a mobile client?

Lab 3: offline-first task manager

- Start by showing cached SQLite tasks.
- Refresh tasks from JSONPlaceholder.
- Save fresh tasks without duplicating IDs.
- Disable network access and reload the screen.
- Display a clear extbfOffline: showing saved data message.
- Recover when connectivity returns.

Deliverable

Demonstrate the same screen online, offline and after reconnecting.

E. Reliability and testing

What happens when the happy path breaks?

Timeouts and transport errors

```dart
import 'dart:async';
import 'dart:io';

try {
  final res = await client.get(uri)
      .timeout(const Duration(seconds: 5));
  if (res.statusCode != 200) {
    throw Exception('HTTP ${res.statusCode}');
  }
} on TimeoutException {
  print('Timed out');
} on SocketException {
  print('Network unavailable');
}
```

A Future timeout stops waiting for the result; it does not necessarily cancel the underlying I/O. Browser builds use different transport exception types.

Four states every async screen needs

Loading

Show progress; prevent accidental duplicate actions.

Success

Show typed data and relevant actions.

Empty

Explain that no records match.

Error

Show an understandable message and Retry.

Discuss. Is an empty task list always an error?

Spot the bug: three common mistakes

```
// 1. New request on every rebuild:
FutureBuilder(future: api.fetchTasks(), builder: ...);

// 2. Context may no longer be mounted:
await repository.load();
Navigator.of(context).pop();

// 3. Fire-and-forget without error handling:
repository.refresh();
```

Try it now. Fix each snippet: stable Future, mounted check, and explicit awaited or handled background operation.

Future.wait and error behavior

- Use `Future.wait()` for independent Futures.
- If one fails, the combined Future completes with an error; other operations are not automatically cancelled.
- The default `eagerError: false` waits for all Futures to finish before reporting the first error.
- If partial results matter, handle each Future independently.

Discuss. Two requests start together; one fails. Does the other network request stop automatically?

Testing the HTTP boundary

```dart
import 'package:http/testing.dart';

final client = MockClient((request) async {
  return http.Response(
    '[{"id":1,"title":"Test","completed":false}]',
    200,
  );
});

final api = TaskApi(client);
final tasks = await api.fetchTasks();
assert(tasks.single.title == 'Test');
```

MockClient avoids real internet dependency. The TaskApi implementation and imports are from earlier slides. Also test 404, malformed JSON and transport failures.

Testing the database boundary

- Use an isolated test database rather than student production data.
- `sqflite_common_ffi` can support SQLite tests on the Dart VM.
- Test insert, query, update, delete and transaction rollback.
- Test duplicate IDs and constraint failures.
- Close the database and clean up after each test.

Discuss. Why is a real in-memory SQLite test useful even when the HTTP layer is mocked?

Repository tests: the important cases

Scenario	Expected outcome
API success	Returned tasks and saved local records
Network failure + cache	Cached tasks displayed with offline state
Network failure + no cache	Useful error or explicit empty/offline state
Invalid JSON	Parsing error handled according to policy
Database failure	No false claim that persistence succeeded
Competing refreshes	Old data does not overwrite newer data

CPU-bound work: when async is not enough

```dart
import 'dart:convert';
import 'package:flutter/foundation.dart';

List<dynamic> parseLargeJson(String text) {
  return jsonDecode(text) as List<dynamic>;
}

final items = await compute(parseLargeJson, largeBody);
```

Rule of thumb

Await I/O. For expensive CPU-bound parsing or processing, consider isolates/compute on supported platforms.

Discuss. Will `await` alone prevent a huge synchronous JSON decode from janking?

Capstone: Student Task Manager

- GET a task list from JSONPlaceholder.
- Decode it into typed Task objects.
- Persist it in SQLite.
- Render cached tasks without internet.
- Add loading, error, empty and retry states.
- Inject a fake HTTP client and test one failure path.

Bonus

Add a refresh epoch guard and explain how you prevent stale writes to SQLite.

Assessment rubric

Criterion	Weight
Correct Futures and async/await	20%
HTTP status handling and JSON models	20%
SQLite CRUD and persistence	20%
Offline-first repository behavior	20%
UI states and recovery	10%
Testability and code organization	10%

Evidence: running app, offline demonstration, and a short explanation of the chosen data-flow policy.

Exit ticket: three questions

1. Does `await` block the whole Flutter UI isolate? Explain.
2. If an HTTP POST returns 201 from JSONPlaceholder, is the record necessarily stored permanently?
3. If SQLite already contains tasks but the network fails, what should the repository return or display?

One-minute reflection

Which failure boundary will you test first in your next app: HTTP, SQLite or the repository combining them? Why?

References and further reading

- Dart asynchronous programming: `https://dart.dev/libraries/async/async-await`
- Dart Futures and Streams: `https://dart.dev/libraries/async`
- Flutter networking: `https://docs.flutter.dev/data-and-backend/networking`
- Flutter SQLite cookbook: `https://docs.flutter.dev/cookbook/persistence/sqlite`
- JSONPlaceholder: `https://jsonplaceholder.typicode.com`
- httpbin: `https://httpbin.org`
- DummyJSON: `https://dummyjson.com`
- Supabase Flutter documentation: `https://supabase.com/docs/guides/getting-started/quickstarts/flutter`

Closing principle: Build for slow networks, missing data and recoverable failures—not only the perfect demo.